RESTAURANT MANAGEMENT SYSTEM

JOE & THE JUICE

SECTION #41209

Team #5

Phase 3

Contents

Overview..... 3
Requirement Specification 4
Entity Relationship Diagram 5
RELATIONAL SCHEMA..... 6
System users 7
Implementation 14
GUI representation 19
Difficulties..... 29

O V E R V I E W

We are creating a database management system for Joe & The Juice restaurant. Our system will clarify and simplify the way everything works for the managers, keeping them updated and aware of the workflow for all branches of the restaurant.

The system will represent managers and employees working in each branch, it will inform the manager about the branch's states and tracks the number of orders for each branch by their locations, it's also giving the ability to determine customer orders and the dishes they choose with the bills of the orders and their total prices

REQUIREMENT SPECIFICATION

The restaurant has many employees, each described by a unique ID, phone number, national address, name, age, email, gender, position, and salary.

The restaurant has many branches. Each branch has a unique code, location (city, neighborhood, street), capacity, and time (opening time, closing time) changes according to the days.

Each branch has at least 4 employees working for it where every employee should work for at least one branch, and every branch should be managed by one employee.

Each branch offers only one menu, which has a unique ID and name. Many branches can have the same menu.

Every menu offers many dishes, every dish has a unique code, price, category, name, description, and calories.

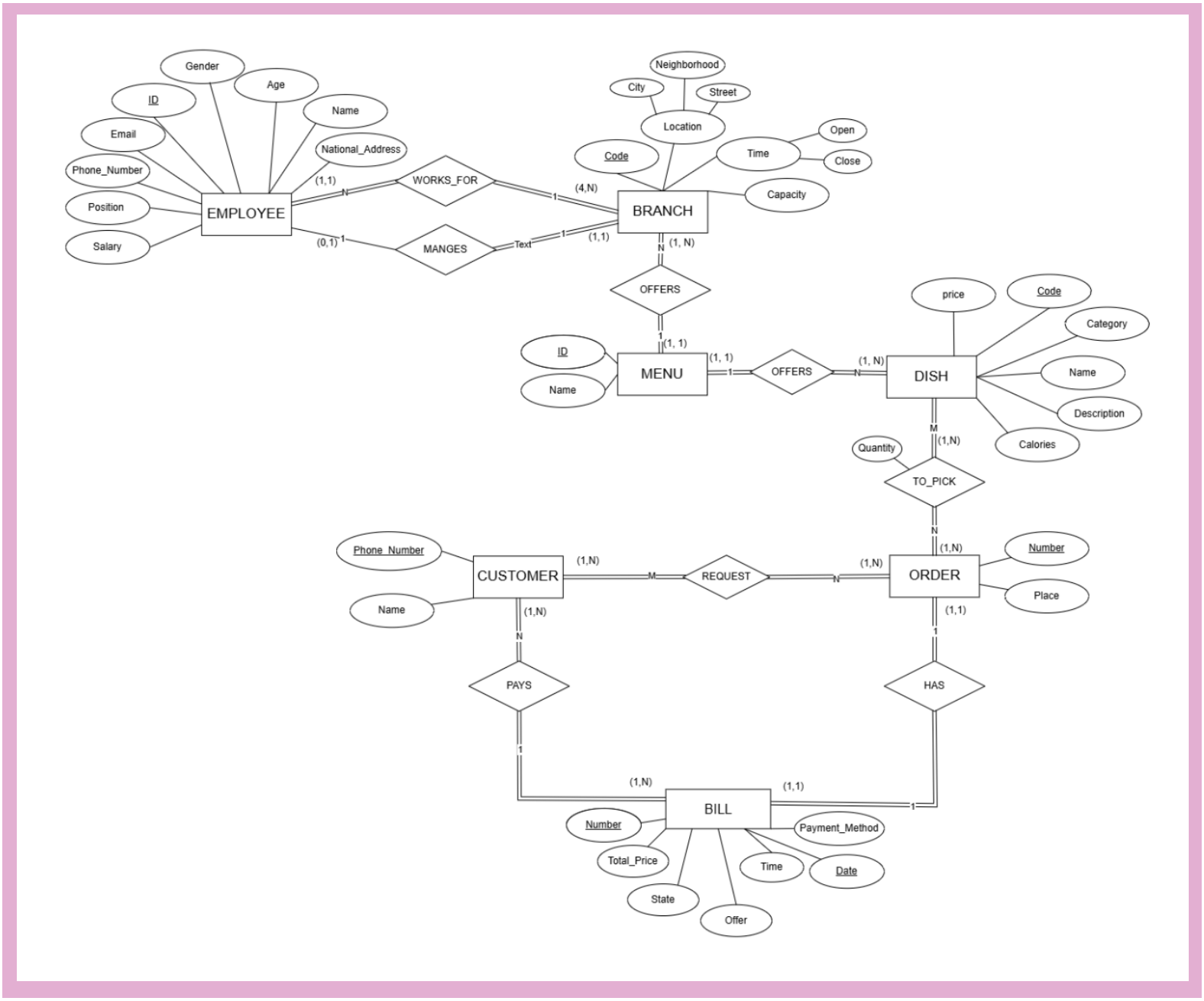
Each order has a unique number, and place. Orders can pick as many as wanted only from dishes and quantity must be placed with the dish, and the dish can be picked for many orders.

Each order has one bill, which has a unique number, total price, state, offer, time described as (hour, minute), unique date and payment method.

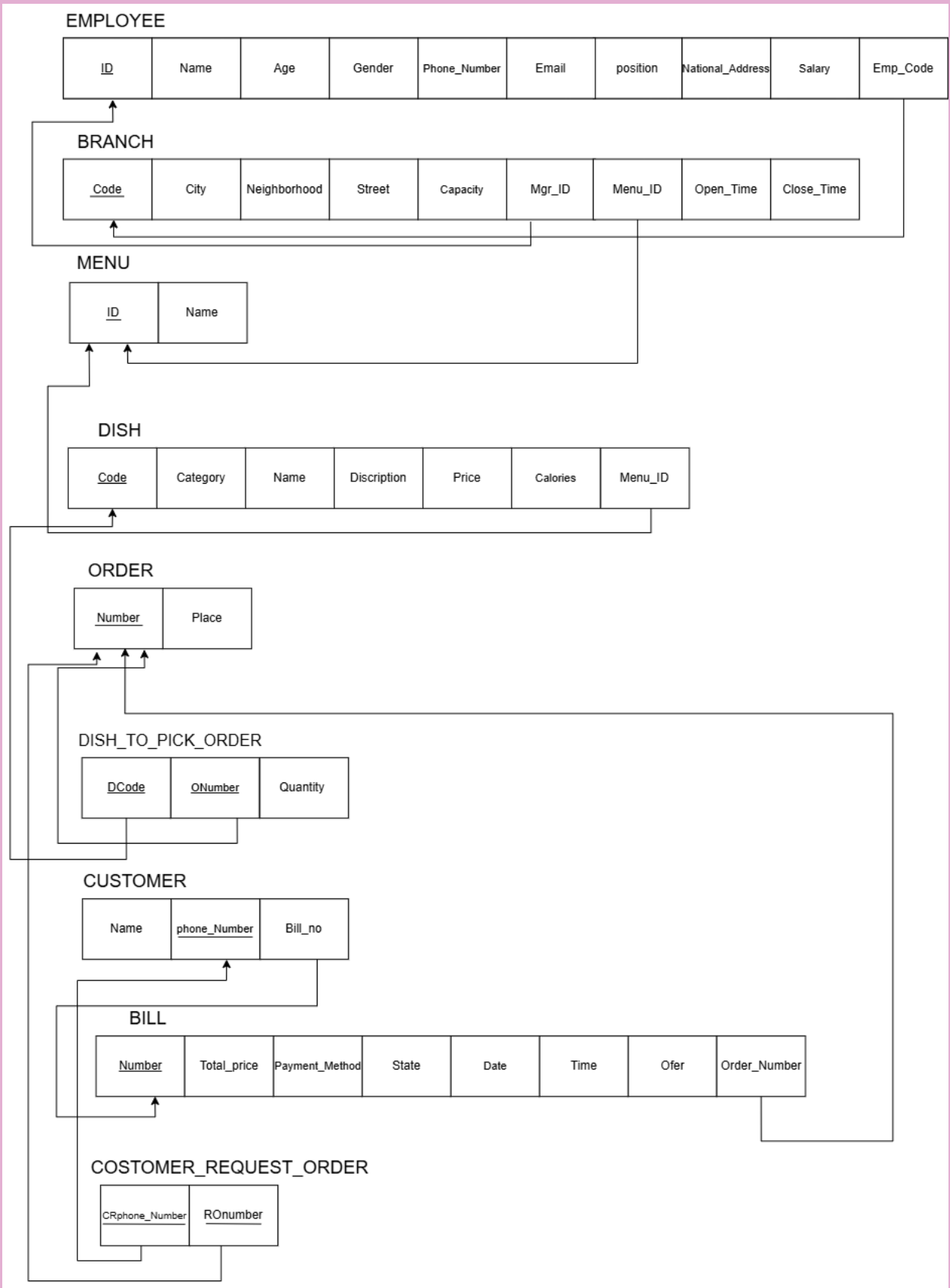
The restaurant has many customers who are described by their name and phone number, each customer must request at least one order.

The customer must pay the bill for his orders,

ENTITY RELATIONSHIP DIAGRAM



RELATIONAL SCHEMA



S Y S T E M U S E R S

MANAGER

The manager's responsibility is to manage. This user group has access to everything related to the branch, including opening and closing times, employee and customer data, sales growth, etc.

The accessible attributes are

EMPLOYEE

ID	Name	Age	Gender	Phone_Number	Email	position	National_Address	Salary	Emp_Code

BRANCH

Code	City	Neighborhood	Street	Capacity	Mgr_ID	Menu_ID	Open_Time	Close_Time

MENU

ID	Name

DISH

Code	Category	Name	Discription	Price	Calories	Menu_ID

ORDER

Number	Place

DISH_TO_PICK_ORDER

DCode	ONumber	Quantity

CUSTOMER

Name	phone_Number	Bill_no

BILL

Number	Total_price	Payment_Method	State	Date	Time	Ofer	Order_Number

COSTOMER_REQUEST_ORDER

CRphone_Number	RONumber

INSERT

```sql
INSERT INTO MENU  (Menu_ID , Menu_Name)
values
   (1, 'Summer Menu'),
   (2, 'Winter Menu');

INSERT INTO BRANCH (Branch_Code , City , Neighborhood , Street , Capacity, Manager_ID, Menu_ID, Open_Time, Close_Time)
values
    (1, 'Riyadh', 'Al-Sulaimaniya', 'King Saud Road', 100,null,1,'07:00:00','01:00:00'),
    (2, 'Riyadh', 'Alaqiq', 'Kafd ring roud', 150,null,1,'07:00:00','01:00:00'),
    (3, 'Riyadh', 'alsafarat', 'Alkhawaba', 100, null,1,'07:00:00','01:00:00');

INSERT INTO EMPLOYEE  (Employee_ID, Employee_Name, Age, Gender, Phone_Number, Email, Employee_position, National_Address, salary, Branch_code)
VALUES
    (12534, 'Lama', 26, 'F', '966599777888', 'lama123@gmail.com', 'Manager', 23456, 12000,1),
    (12536, 'Hanin', 20, 'F', '966599899399', 'hanin123@gmail.com', 'Chef', 12345, 10000,1),
    (12537, 'Nouf', 30, 'F', '966599666777', 'nouf_barista@gmail.com', 'Barista', 34567, 9000,1),
    (12540, 'khaled', 25, 'M', '966599555666', 'khaled123@gmail.com', 'Cashier', 45678, 8500,1),
    (12535, 'Fay', 26, 'F', '966599777253', 'fay_123@gmail.com', 'Manager', 45363, 12000,2),
    (12539, 'Yasser', 20, 'M', '966599899334', 'yasser123@gmail.com', 'Chef', 97865, 10000,2),
    (12538, 'Yara', 30, 'F', '966599666745', 'yara353@gmail.com', 'Barista', 12323, 9000,2),
    (12541, 'Nawaf', 25, 'M', '966599555667', 'Nawaf45@gmail.com', 'Cashier', 98675, 8500,2),
    (12522, 'Ahmed', 28, 'M', '966598888345', 'ahmed_123@gmail.com', 'Manager', 34521, 12000,3),
    (12523, 'Omar', 35, 'M', '966599444963', 'omar_chef@gmail.com', 'Chef', 76859, 10000,3),
    (12524, 'Demma', 30, 'F', '966599666711', 'domo@gmail.com', 'Barista', 49374, 9000,3),
    (12525, 'noura', 24, 'F', '966599555634', 'noura123@gmail.com', 'Cashier', 98743, 8500,3);


UPDATE BRANCH
SET Manager_ID = 12535
WHERE Branch_Code = 2;

UPDATE BRANCH
SET Manager_ID = 12522
WHERE Branch_Code = 3;


UPDATE BRANCH
SET Manager_ID = 12534
WHERE Branch_Code = 1;

INSERT INTO DISH  (Dish_code ,Category, Dish_Name, Dish_description, Price, Calories,Menu_ID)
values
   (1, 'Sandwich', 'Spicy Tuna', 'Tuna, Tomato, Jalapenos, Tabasco, Pesto', 29, 525,1),
   (2, 'Sandwich', 'Avocado', 'Avocado, Modzarella, Tomatos, Pesto ', 29, 485,1),
   (3, 'Sandwich', 'Joes Club', 'Chicken, Avocado, Tomatos, Pesto', 29, 448,1),
   (4, 'Sandwich', 'Tunacado', 'Tuna, Avocado, Tomato, Pesto', 29, 571,2),
   (5, 'Sandwich', 'Turkey', 'Turky, Mozzarella, Tomato, Pesto', 29,497,2),
   (6,'Sandwich', 'Spaicy Club', 'chicken, avocado,tomatos, pesto', 29, 451,2),
   (7, 'Sandwich', 'Halloumi', 'chicken, avocado,tomatos, pesto', 29, 642,2),
   (8, 'Sandwich', 'Bresaola', 'chicken, avocado,tomatos, pesto', 29, 411,1),
   (9, 'Salad Bowl', 'Green tuna ', 'Lemon, Spinach, Cranberry, Pumpkin Seeds, Pickled Red Onion, Edamame, Tuna, Cucumber, Kale, Pesto ', 36, 560,1),
   (10, 'protein Shakes', 'Beets & Berries', 'Strawberries, Beetroot Powder, Banana, Date Puree, Ice, Whey Protein', 33, 370,1),
   (11, 'shakes', 'Power Shake', 'Vanilla Milk, Strawberries, Banana, Ice', 24,290,1 ),
   (12, 'Shakes', 'Acia', 'Coconut Drink, Avocado, Banana, Peanut Butter, Ice, Acai', 29, 360,1),
   (13, 'Juices', 'GO Away DOC', 'Carrot, Ginger,Apple,Olive Oil, Ice', 29, 177,1),
   (14, 'Juices', 'Green Tonic', 'Kale, Celert, Cucumber, Olive', 29, 70,1),
   (15, 'Coffe', 'Pink late', 'Regular Mile, Espresso, ', 18, 243,1),
   (16, 'Coffe', 'Iced Vanilla Matcha', 'Vanilla Syrup, Matcha Regular Milk, Ice', 24, 204,1),
   (17, 'Snacks & Treats', 'Blueberry Muffin', 'Blueberry Muffin', 16, 306,1),
   (18, 'Shots', 'Ginger Shot', 'Apple, Ice, Ginger', 10, 25,1);
```

UPDATE

```
UPDATE EMPLOYEE
SET Employee_position = 'Manager'
WHERE Employee_ID = '12536';

UPDATE EMPLOYEE
SET Salary = 12000
WHERE Employee_ID = 12536;
```

DELETE

```
DELETE FROM Employee
WHERE Employee_ID = 12541 ;

DELETE FROM Dish
WHERE Dish_code =15 ;
```

RETRIEVE

One table

- **Retrieve the name and position of employees who are older than 30.**
- **Retrieve the salary of the employee, whose name is Lama.**

```
SELECT Employee_Name, Age, Employee_position
FROM EMPLOYEE
WHERE Age < 30;

SELECT Salary
FROM  EMPLOYEE
WHERE Employee_Name = 'Lama';
```

Tow table

- **Retrieve the names, positions, the city they work at, and the capacity of their branch**

```
SELECT e.Employee_Name, e.Employee_Position, b.City, b.Capacity
FROM  EMPLOYEE e
JOIN BRANCH b ON e.Branch_Code = b.Branch_Code;
```

Three tables

- **Retrieve the names and positions of female employees who work at branch code 1.**

```sql
SELECT
    e.Employee_Name,
    e.Employee_Position
FROM  EMPLOYEE  e
JOIN  Branch b ON e. Branch_Code = b.Branch_Code
WHERE
    b.Branch_Code = 1
    AND e.Gender = 'F';
```

EMPLOYEE

The employee's responsibility is to add the order from the menu, add a dish, and add customer information

The accessible attributes are

ORDER

Number	Place

DISH_TO_PICK_ORDER

DCode	ONumber	Quantity

CUSTOMER

Name	phone_Number	Bill_no

BILL

Number	Total_price	Payment_Method	State	Date	Time	Ofer	Order_Number

COSTOMER_REQUEST_ORDER

CRphone_Number	ROnumber

INSERT

```sql
INSERT INTO CUSTOMER (Phone_Number,Customer_Name)
VALUES
 ( 565653656 ,'lolo',1),
 ( 565654656,'fahad',2);

INSERT INTO ORDERS
VALUE
(1, 'dine_in'),
(2, 'dine_in'),
(3, 'dine_in');

INSERT INTO DISH_TO_PICK_ORDER
VALUES
(1, 1, 2),
(2, 2, 1);
```

UPDATE

```sql
UPDATE ORDERS
SET place= 'dine_out'
WHERE Order_Number =1 ;

UPDATE DISH_TO_PICK_ORDER
SET  Quantiti = 1
WHERE  Order_Number =1 ;
```

DELETE

```sql
DELETE FROM Dish
WHERE Dish_code =15 ;

DELETE FROM orders
WHERE Order_Number =3 ;
```

RETRIEVE

One table

- **Retrieve the name of the customer whose phone number 565653656**
- **Retrieve the menu**

```
SELECT Customer_Name
FROM Customer
WHERE Phone_Number = 565653656 ;
```

```
SELECT *
FROM Menu ;
```

Tow table

- **Retrieve the payment method for the customer lolo**

```
SELECT b.Payment_Method
FROM Customer c
JOIN Bill b ON c.Phone_Number = b.Customer_Phone
WHERE c.Customer_Name = 'lolo';
```

Three tables

- **Retrieve the name for the customer who ordered Green tonic juice**

```
SELECT c.Customer_Name
FROM Customer c
JOIN Orders o ON c.Phone_Number = o.Order_Number
JOIN DISH_TO_PICK_ORDER od ON o.Order_Number = od.Order_Number
JOIN Dish d ON od.Dish_Code = d.Dish_code
WHERE d.Dish_Name = 'Green Tonic';
```

I M P L E M E N T A T I O N

This is a critical part of our Java code, though not the entire program.

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class mainSQL {

    private static final String URL ="jdbc:mysql://127.0.0.1:3306/JoeAndTheJoes";
    private static final String USER = "root"; // Replace with your MySQL username
    private static final String PASSWORD = "fayDBcs5?"; // Replace with your MySQL password

    public static Connection getConnection() {
        Connection connection = null;
        try {
            connection = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("Database connected!");
        } catch (SQLException e) {
            e.printStackTrace();
            System.out.println("Failed to connect to the database.");
        }
        return connection;
    }

 public static void main(String[] args) {
        Connection conn = getConnection();
        if (conn != null) {
            System.out.println("Connection successful!");
        }
    }
}
```

This code ensures that our SQL database is properly connected to the GUI.

```java
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
  try {
        int enteredID = Integer.parseInt(jTextField3.getText().trim());

        String position = getEmployeePosition(enteredID);

        if (position != null) {
            if (position.equalsIgnoreCase("Manager")) {

                Frame1 frame = new Frame1();
                frame.setVisible(true);
                this.dispose();
            } else {

                Frame4 frame = new Frame4();
                frame.setVisible(true);
                this.dispose();
            }
        } else {
            JOptionPane.showMessageDialog(this, "ID not found! Please try again.", "Error", JOptionPane.ERROR_MESSAGE);
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Please enter a valid numeric ID!", "Invalid Input", JOptionPane.WARNING_MESSAGE)
    }
```

After the user enters their ID, the code checks if the ID exists. If it does, the system searches for the user's position. If the user is a manager, they will be directed to the manager frame; if they are an employee, they will be directed to the employee

```java
private void jComboBox1ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String selectedOption = (String) jComboBox1.getSelectedItem();

    // Hide all panels first
    AddEmpPanel.setVisible(false);
    RemoveEmpPanel.setVisible(false);
    ModifyEmpPanel.setVisible(false);
    ShowEmpPanel.setVisible(false);

    // Show the panel based on the selected option
    switch (selectedOption) {
        case "Add Employee":
            AddEmpPanel.setVisible(true);
            break;
        case "Remove Employee":
            RemoveEmpPanel.setVisible(true);
            break;
        case "Modify Employee":
            ModifyEmpPanel.setVisible(true);
            break;
        case "Show Employee":
            ShowEmpPanel.setVisible(true);
            loadEmployees(); // Load data into the table when the panel is shown
            break;
    }
}
```

This part of the code allows the manager to choose a panel between: "Add Employee", "Remove Employee", "Modify Employee", or "Show Employee".

```java
private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:

try (Connection connection = mainSQL.getConnection()) {
    int id = Integer.parseInt(jTextField10.getText().trim());

    String query = "DELETE FROM EMPLOYEE WHERE Employee_ID = ?";
    PreparedStatement statement = connection.prepareStatement(query);
    statement.setInt(1, id);

    int rowsDeleted = statement.executeUpdate();
    if (rowsDeleted > 0) {
        JOptionPane.showMessageDialog(this, "Employee removed successfully!", "Success", JOptionPane.INFORMATION_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "No employee found with the given ID.", "Error", JOptionPane.ERROR_MESSAGE);
    }
} catch (Exception e) {
    JOptionPane.showMessageDialog(this, "Error removing employee: " + e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
}

}
```

If the employee removal is successful, a confirmation message will appear. If not, it indicates that the employee was not found, and an error message will be displayed.

```java
private void jButton6ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
String orderNumber = jTextField14.getText(); // Order Number
String billNumber = jTextField15.getText();  // Bill Number
String offerText = jTextField7.getText();   // Offer/Discount
String paymentMethod = jCheckBox3.isSelected() ? "Credit Card" : jCheckBox4.isSelected() ? "Cash" : null;

if (orderNumber.isEmpty() || billNumber.isEmpty() || paymentMethod == null) {
    JOptionPane.showMessageDialog(this, "Please fill in all required fields and select a payment method.");
    return;  }
double offer = offerText.isEmpty() ? 0.0 : Double.parseDouble(offerText); // Default offer is 0.0
boolean state = true; // Set state to true if one of the checkboxes is selected

try (Connection conn = mainSQL.getConnection()) {
    String insertBillSQL = """
        INSERT INTO BILL (Bill_Number, Payment_method, State, Offer, Order_Number, Bill_Time, Bill_Date)
        VALUES (?, ?, ?, ?, ?, CURRENT_TIME(), CURRENT_DATE())
    """;
    PreparedStatement insertBillStmt = conn.prepareStatement(insertBillSQL);
    insertBillStmt.setInt(1, Integer.parseInt(billNumber)); // Bill Number
    insertBillStmt.setString(2, paymentMethod);            // Payment Method
    insertBillStmt.setBoolean(3, state);                  // State
    insertBillStmt.setDouble(4, offer);                   // Offer
    insertBillStmt.setInt(5, Integer.parseInt(orderNumber)); // Order Number
    insertBillStmt.executeUpdate();

    String getTotalPriceSQL = "SELECT Total_price FROM BILL WHERE Bill_Number = ?";
    PreparedStatement getTotalPriceStmt = conn.prepareStatement(getTotalPriceSQL);
    getTotalPriceStmt.setInt(1, Integer.parseInt(billNumber));
    ResultSet rs = getTotalPriceStmt.executeQuery();

    if (rs.next()) {
        double totalPrice = rs.getDouble("Total_price");
    } else {
        JOptionPane.showMessageDialog(this, "Bill added, but failed to retrieve total price."); }
} catch (Exception e) {
    e.printStackTrace();
    JOptionPane.showMessageDialog(this, "Error adding bill: " + e.getMessage()); }
}
```

The code handles inserting a new bill into the database by validating inputs, executing an SQL INSERT query, calculating the total price, and displaying appropriate success or error messages.

```java
private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
try (Connection conn = mainSQL.getConnection()) {
    if (jTextField2.getText().isEmpty() || jTextField1.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please provide both Menu ID and Menu Name!");
        return;
    }

    // SQL query to update the Menu table
    String sql = "UPDATE MENU SET Menu_Name = ? WHERE Menu_ID = ?";
    PreparedStatement stmt = conn.prepareStatement(sql);

    int menuId = Integer.parseInt(jTextField2.getText()); // Menu_ID
    String menuName = jTextField1.getText(); // Menu_Name

    stmt.setString(1, menuName);
    stmt.setInt(2, menuId);

    int rowsUpdated = stmt.executeUpdate();
    if (rowsUpdated > 0) {
        JOptionPane.showMessageDialog(this, "Menu updated successfully!");
    } else {
        JOptionPane.showMessageDialog(this, "No menu found with the given ID!");
    }
    displayMenuAndDishesGroupedByCategory();
} catch (Exception e) {
    e.printStackTrace();
    JOptionPane.showMessageDialog(this, "Error updating menu: " + e.getMessage());
}

}
```

For the manager, if they wish to update the menu, they must enter the menu ID and name. If the update is successful, a message will confirm it. If there is an error or if no menu is found, an error message will be displayed in either case.

```java
private void displayMenuAndDishesGroupedByCategory() {
    try (Connection conn = mainSQL.getConnection()) {
        String sql = """
            SELECT M.Menu_ID, M.Menu_Name, D.Category, D.Dish_Name, D.Dish_Description,
                   D.Price, D.Calories
            FROM MENU M
            LEFT JOIN DISH D ON M.Menu_ID = D.Menu_ID
            ORDER BY M.Menu_ID, D.Category;
        """;
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery();

        jTextAreaMenu.setFont(new java.awt.Font("Impact",java.awt.Font.PLAIN,12));
        jTextAreaMenu.setText("");

        int currentMenuId = -1;
        String currentCategory = "";

        while (rs.next()) {
            int menuId = rs.getInt("Menu_ID");
            String menuName = rs.getString("Menu_Name");
            String category = rs.getString("Category");
            String dishName = rs.getString("Dish_Name");
            String description = rs.getString("Dish_Description");
            double price = rs.getDouble("Price");
            double calories = rs.getDouble("Calories");

            if (menuId != currentMenuId) {
                currentMenuId = menuId;
                jTextAreaMenu.append("Menu ID: " + menuId + " - " + menuName + "\n");
                jTextAreaMenu.append("==================================\n");
                currentCategory = "";
            }
```

The code retrieves and displays menus and their dishes grouped by category, using an SQL query with a LEFT JOIN, ordering by Menu_ID and Category, and dynamically formatting the results in a JTextArea.

```java
            if (menuId != currentMenuId) {
                currentMenuId = menuId;
                jTextAreaMenu.append("Menu ID: " + menuId + " - " + menuName + "\n");
                jTextAreaMenu.append("==================================\n");
                currentCategory = "";
            }

            if (category != null && !category.equals(currentCategory)) {
                if (!currentCategory.isEmpty()) {
                    jTextAreaMenu.append("  ----------------------------------\n");
                }
                currentCategory = category;
                jTextAreaMenu.append("  Category: " + category + "\n "); }

            if (dishName != null) {
                jTextAreaMenu.append("    Dish: " + dishName + "\n");
                jTextAreaMenu.append("      Description: " + description + "\n");
                jTextAreaMenu.append("      Price: " + price + "\n");
                jTextAreaMenu.append("      Calories: " + calories + "\n");
            }  }
        } catch (Exception e) {
            e.printStackTrace();
            JOptionPane.showMessageDialog(this, "Error loading menu and dishes: " + e.getMessage());
        }
    }
```

The continuation of the previous code.

GUI REPRESENTATION

Figure 1: After login, the system shows the manager's or employee's view based on the ID.

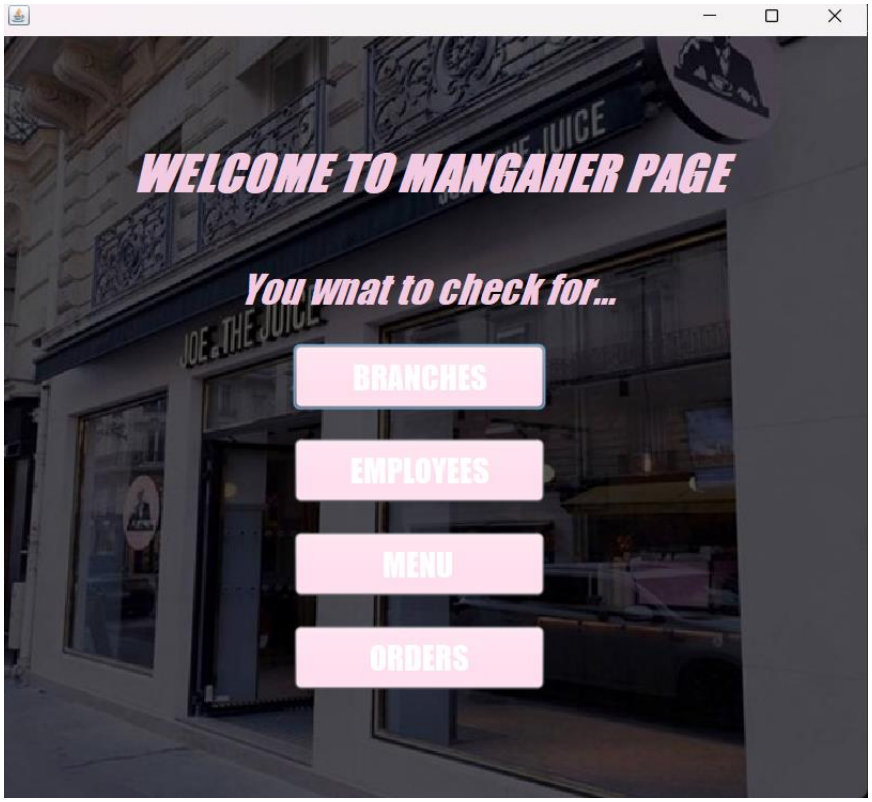
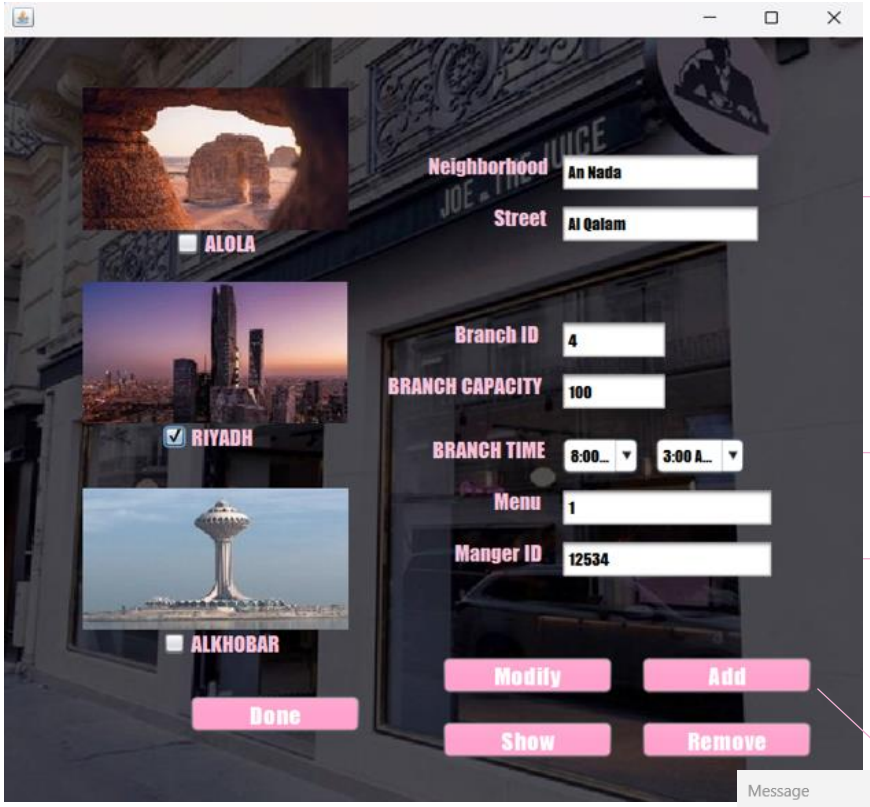


Figure 2: the manager's page (Presses the branch button).



Location

Number of customers in the branch

Open ،close

who will run the branch

Figure 3: the branch's page.

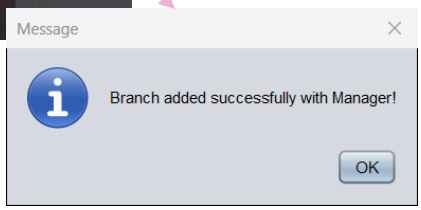


Figure 4: If the branch is added successfully.

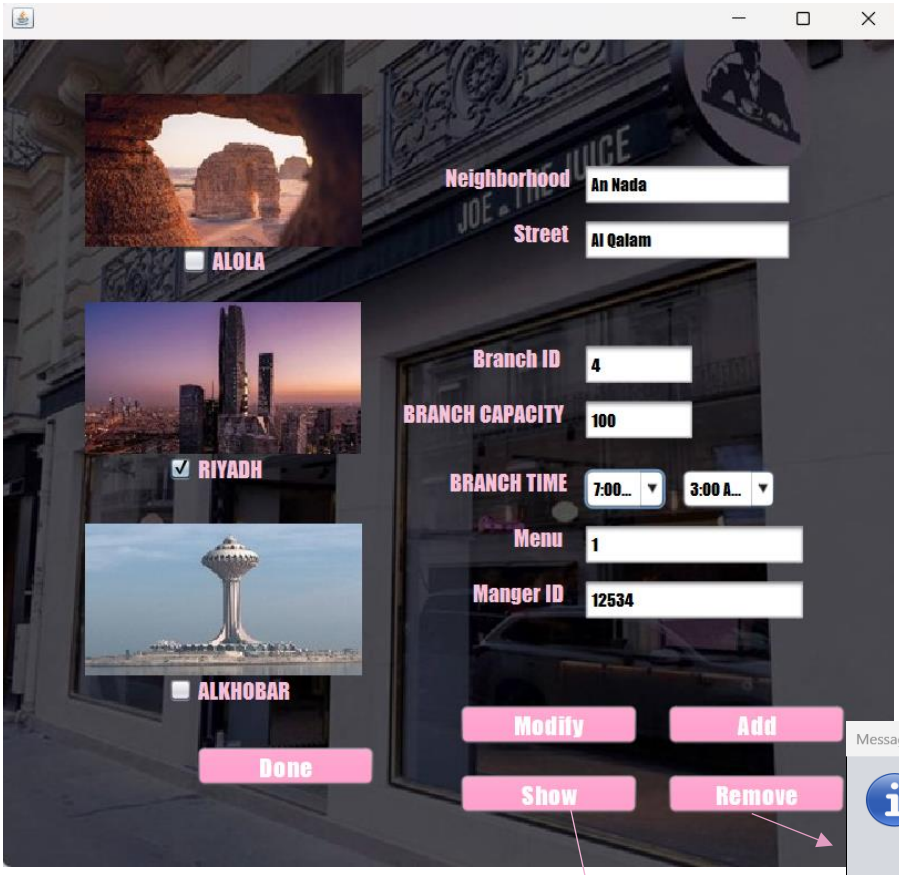


Figure5: the branch's page.

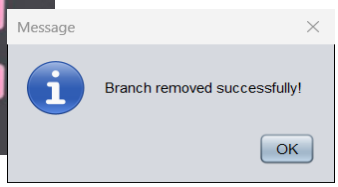


Figure6: If branch removed

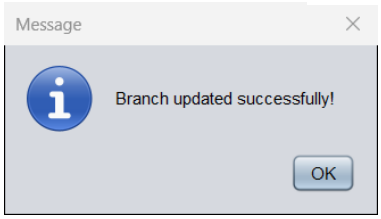


Figure7: If branch updated successfully.

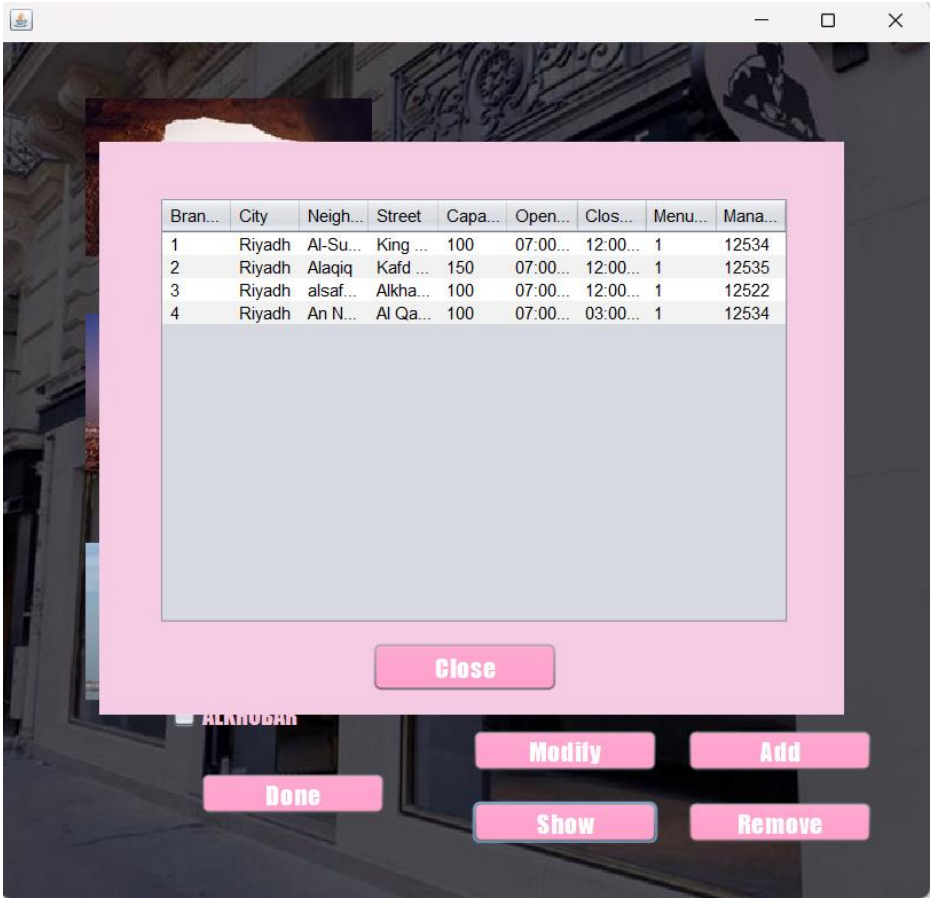
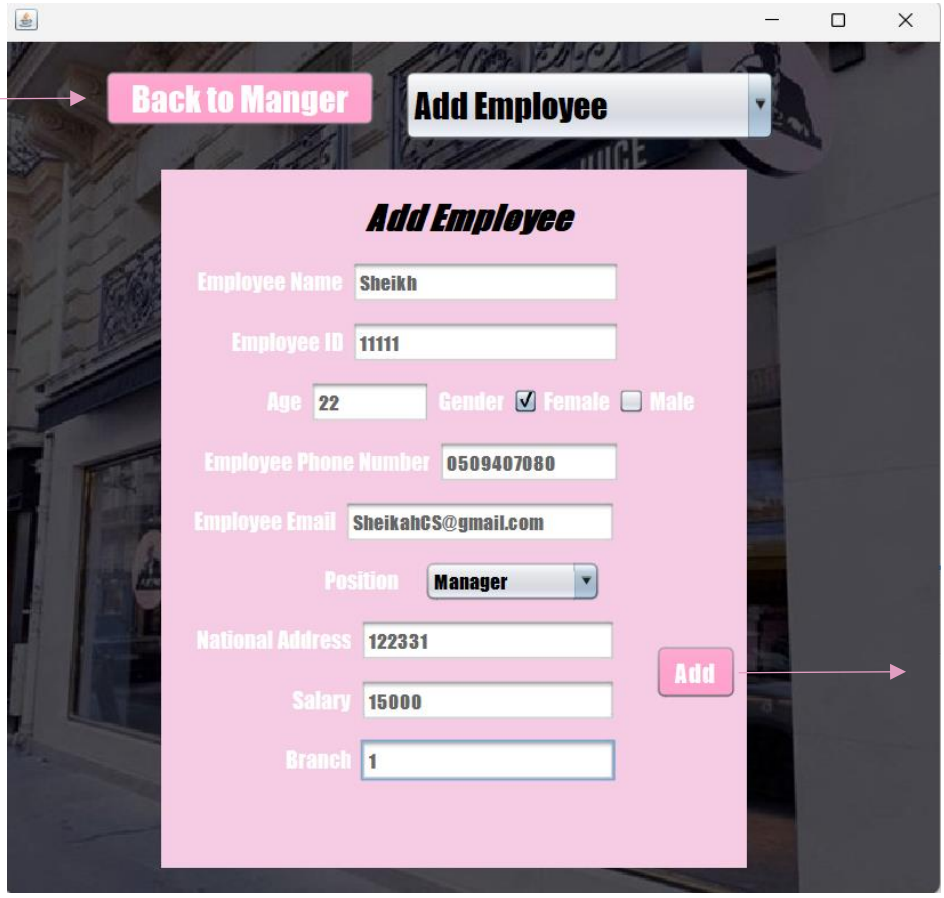


Figure 8: when click "show" button

Back to the "manger's page in [Figure 4]



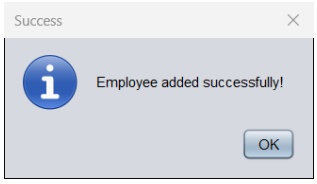


Figure10: If the addition is successful after adding the information

.Figure9: The employee's page.

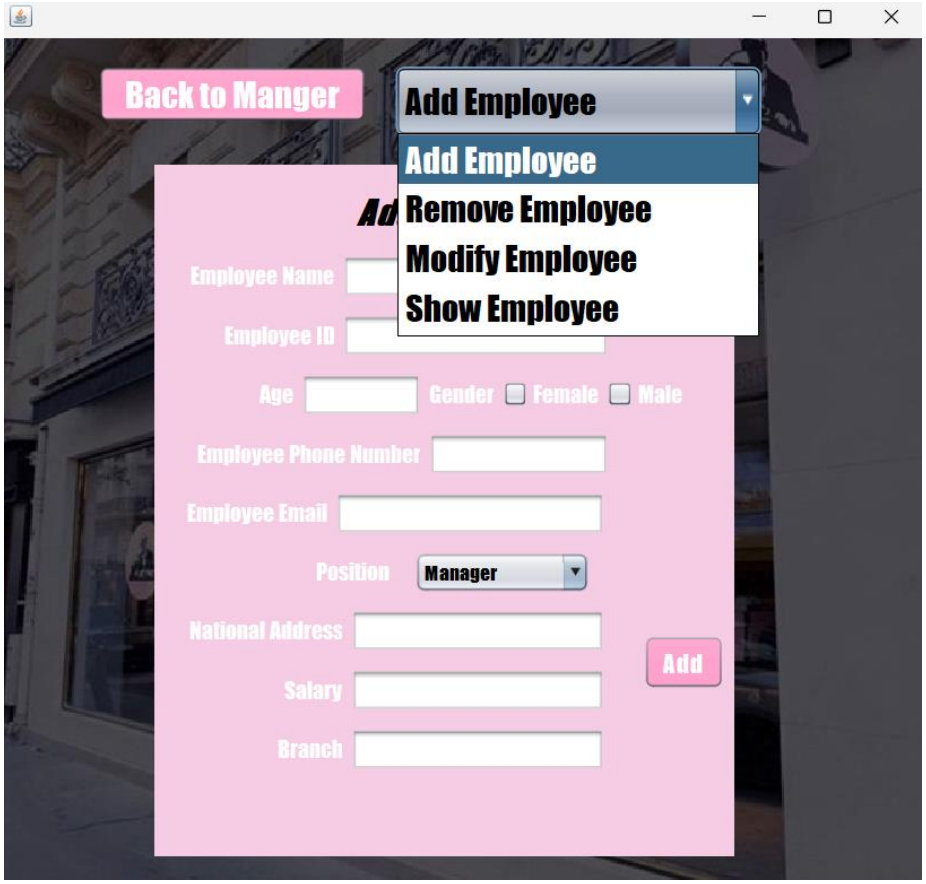
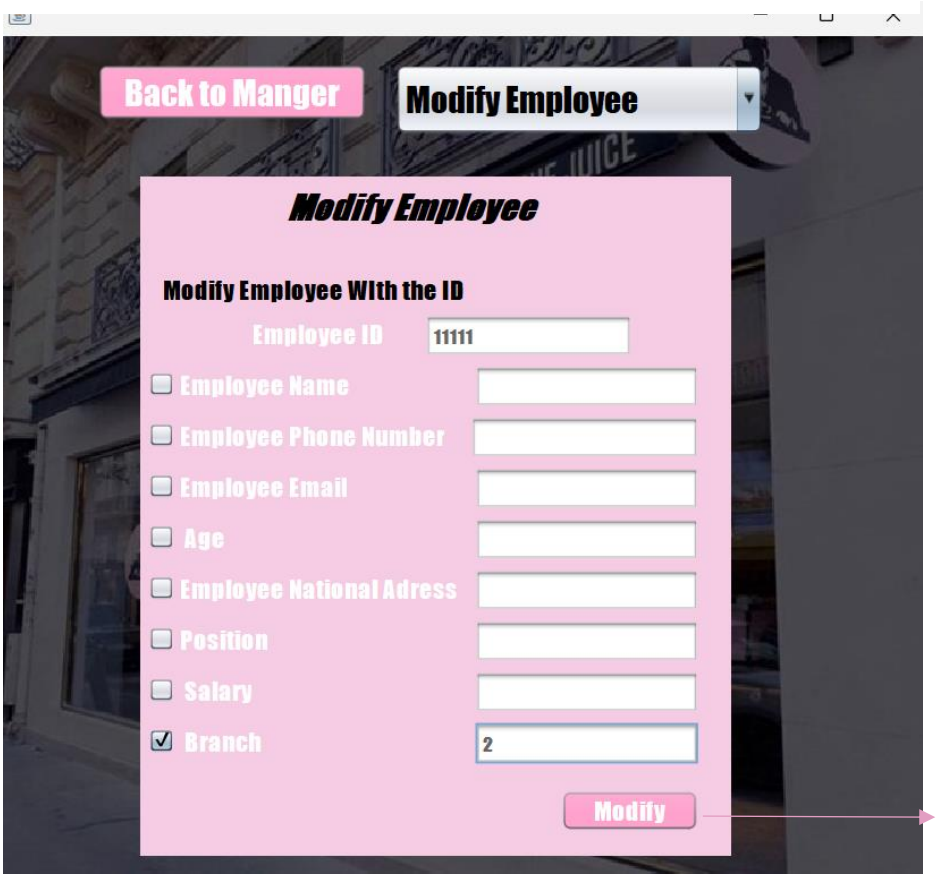


Figure11: The user can change the command he wants to do using combo box.



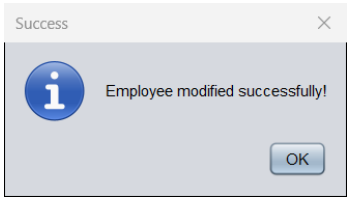
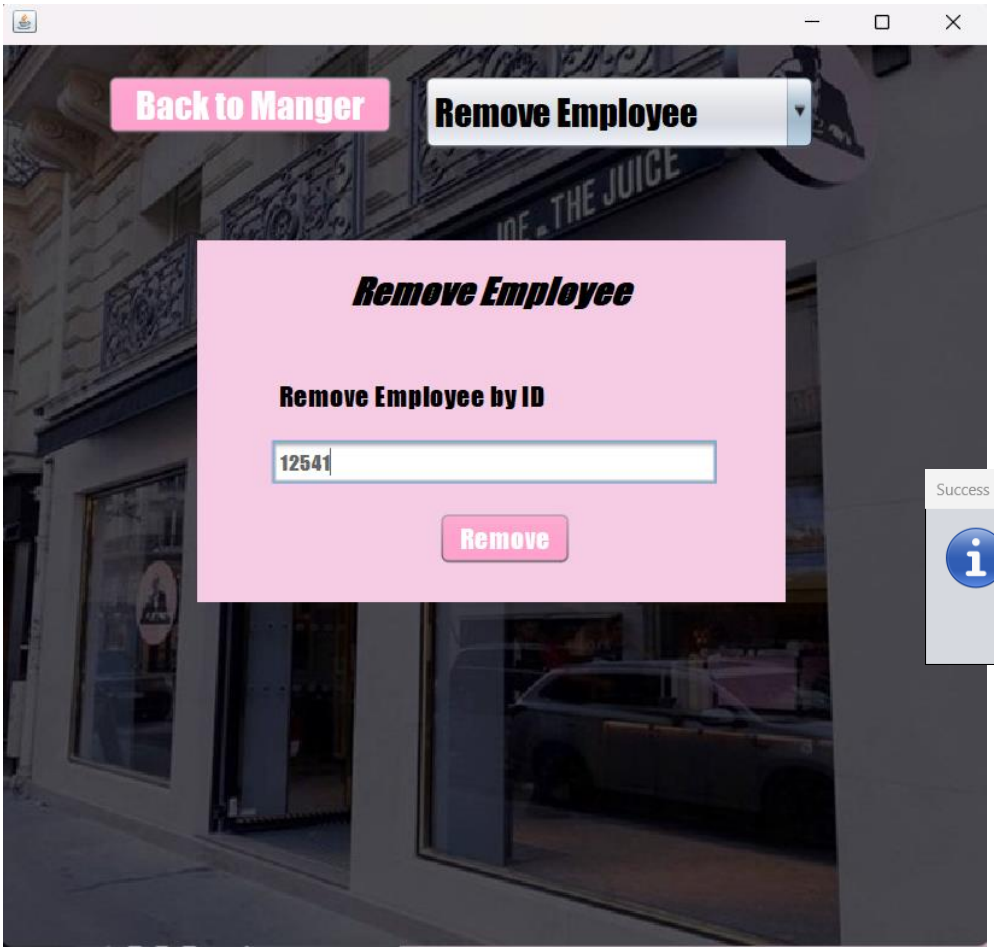


Figure12: The employee's page.

Figure13: If the modify is successful after adding the information



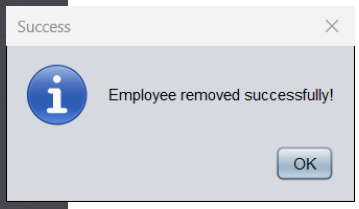
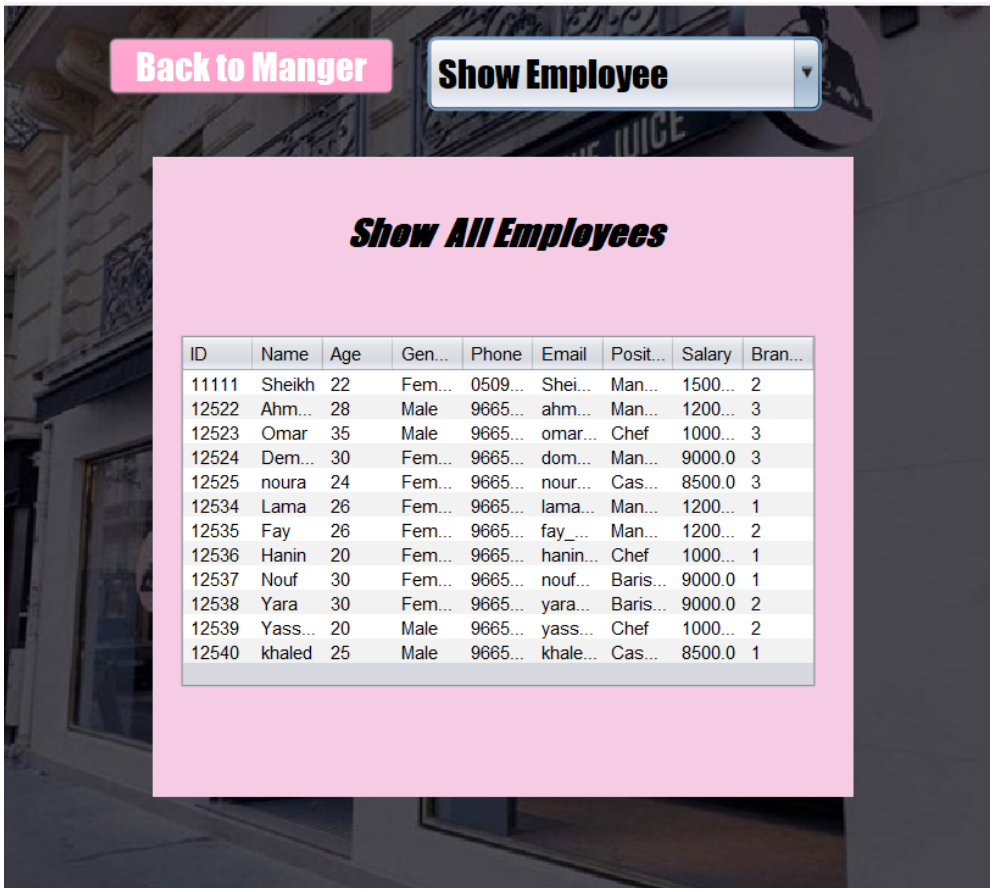


Figure15: If the ID exists

Figure14: The Remove page.



ID	Name	Age	Gen...	Phone	Email	Posit...	Salary	Bran...
11111	Sheikh	22	Fem...	0509...	Shei...	Man...	1500...	2
12522	Ahm...	28	Male	9665...	ahm...	Man...	1200...	3
12523	Omar	35	Male	9665...	omar...	Chef	1000...	3
12524	Dem...	30	Fem...	9665...	dom...	Man...	9000.0	3
12525	noura	24	Fem...	9665...	nour...	Cas...	8500.0	3
12534	Lama	26	Fem...	9665...	lama...	Man...	1200...	1
12535	Fay	26	Fem...	9665...	fay_...	Man...	1200...	2
12536	Hanin	20	Fem...	9665...	hanin...	Chef	1000...	1
12537	Nouf	30	Fem...	9665...	nouf...	Baris...	9000.0	1
12538	Yara	30	Fem...	9665...	yara...	Baris...	9000.0	2
12539	Yass...	20	Male	9665...	yass...	Chef	1000...	2
12540	khaled	25	Male	9665...	khale...	Cas...	8500.0	1

Figure16:

when choosing "show employee" the employee informations will appear

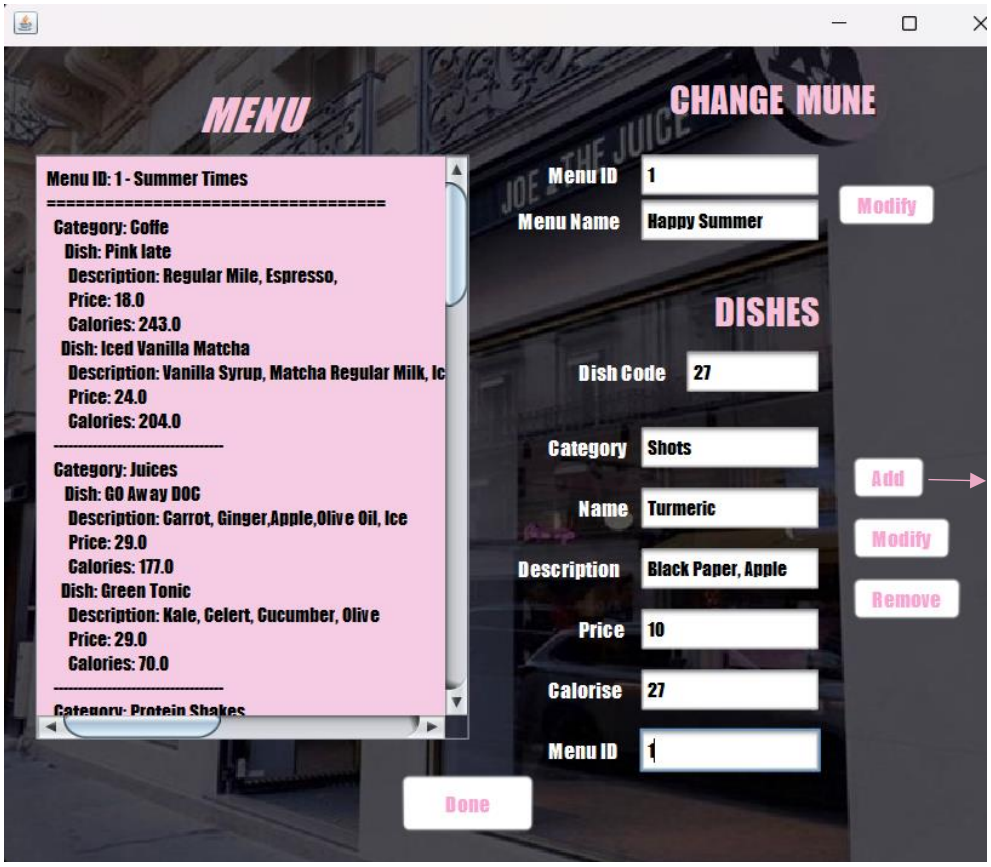


Figure17:

when the manager chooses "menu"

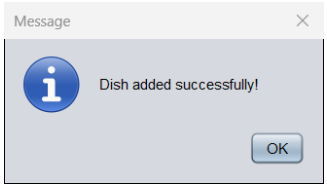


Figure18::

when press the "add" button

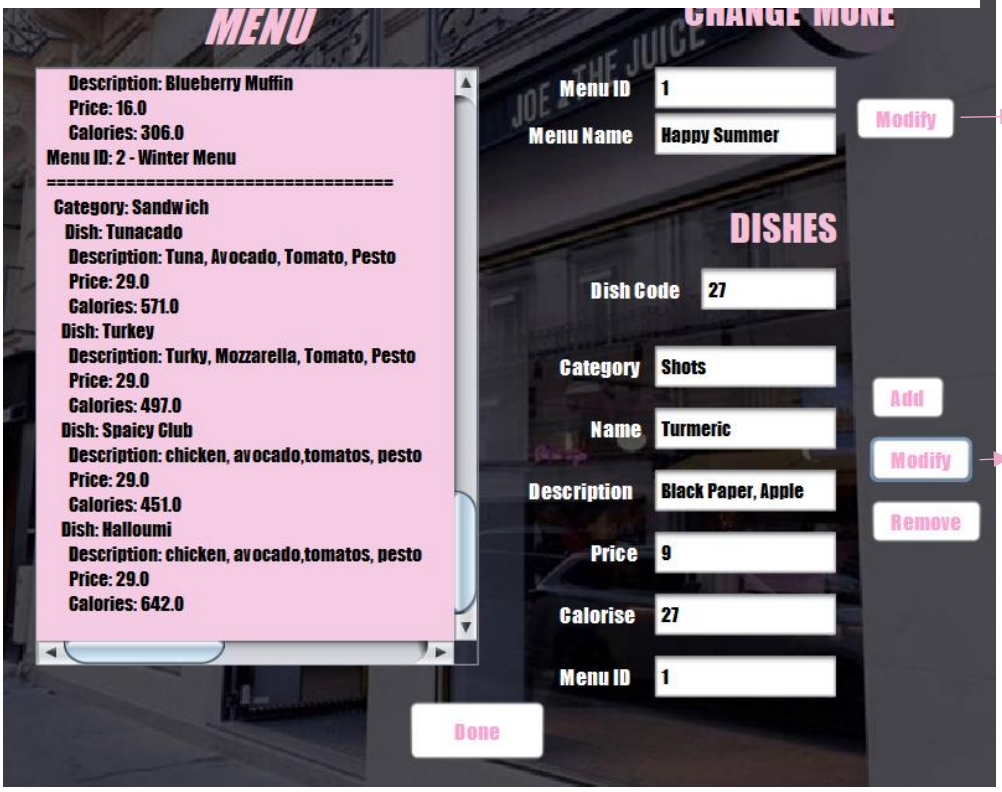


Figure 19: update the menu and dishes

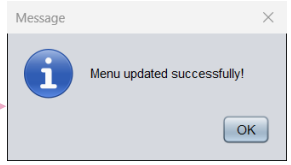


Figure 21: when press the "done" button.

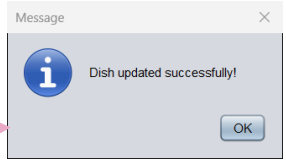


Figure20: when press the "modify ' "button

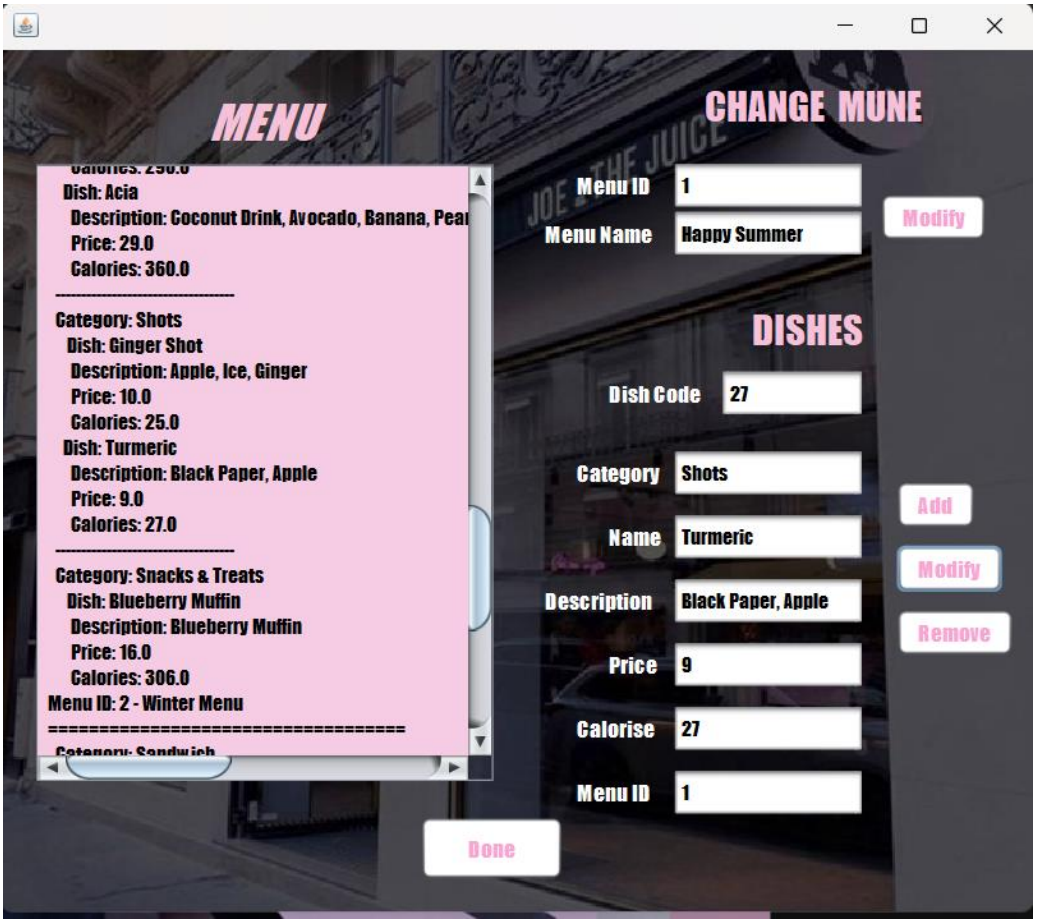


Figure22: the menu after the update

Figure 23: new user will be an Employee now.

Dishes based on category

Quantity

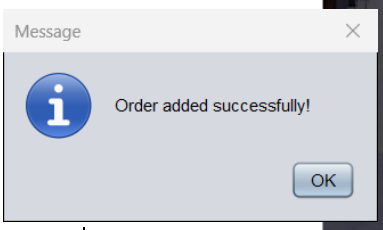


Figure 24: when press the "add" button

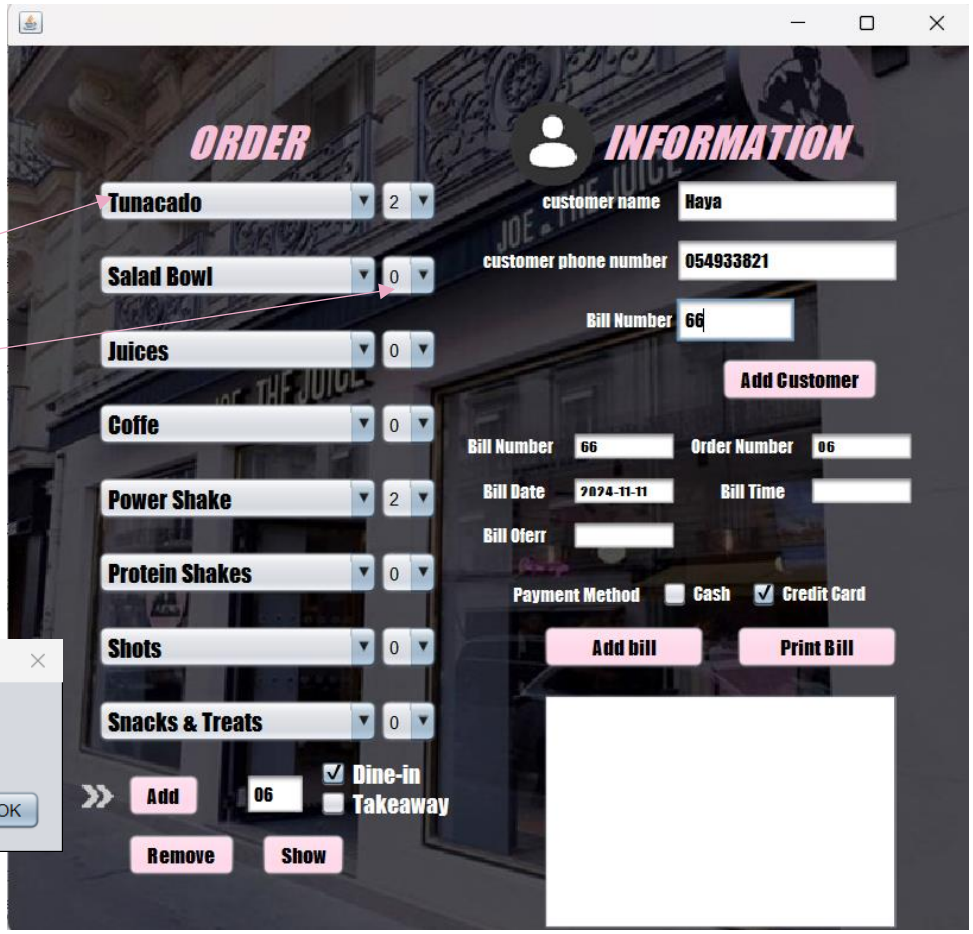
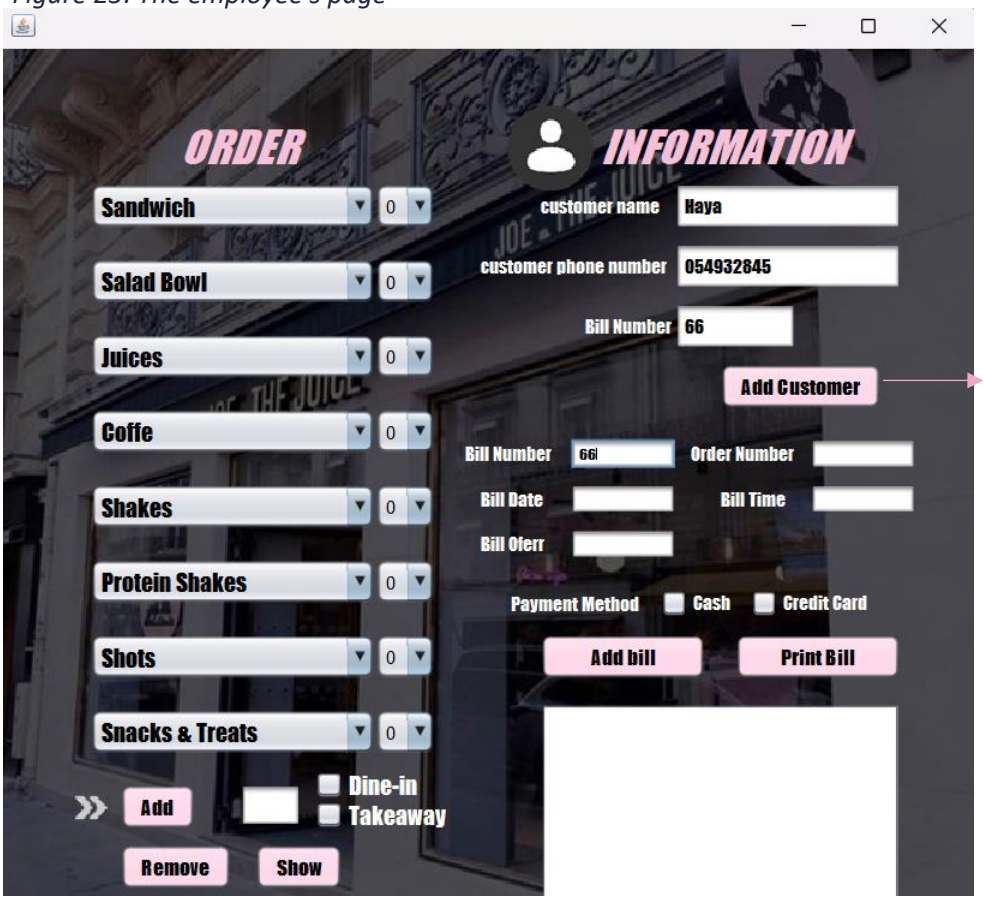


Figure 25: The employee's page



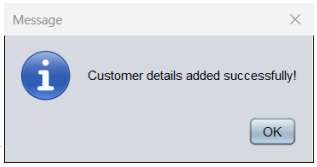


Figure 27: when press the "add customer "button.

Figure 26: Adding the Information on the customer's order

Figure 28: when press the "show" button it shows the order details

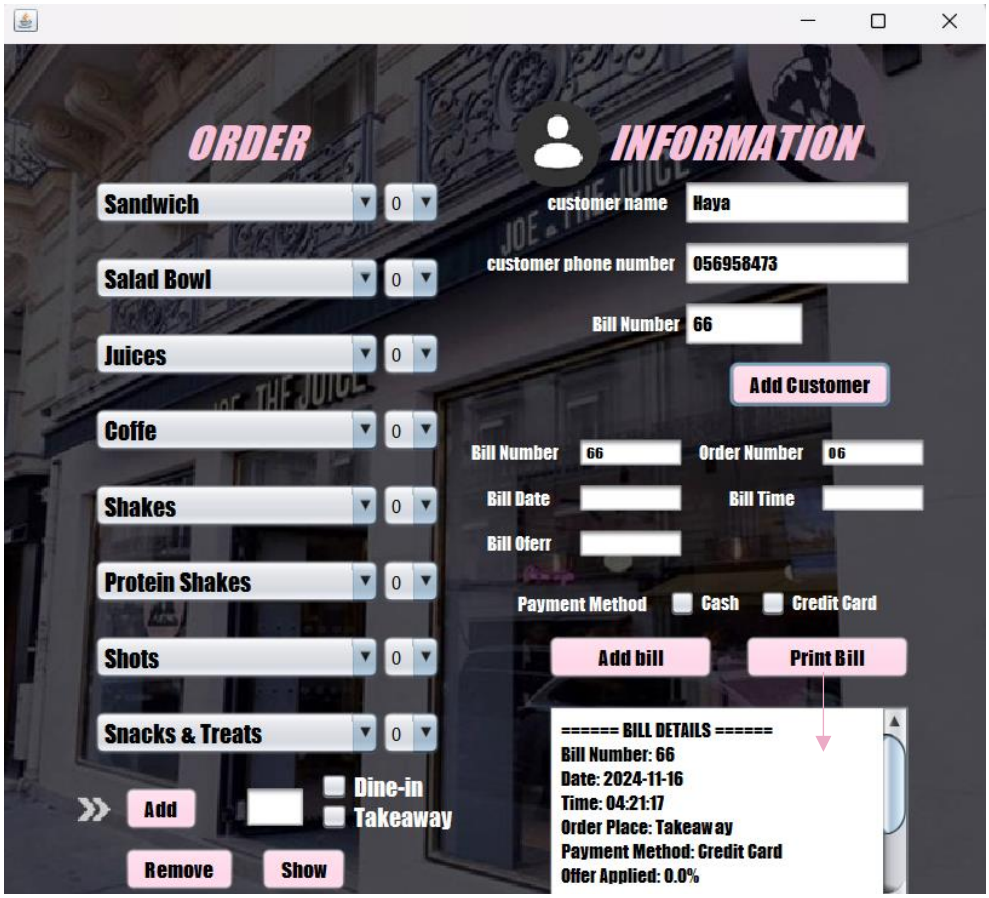


Figure 29: When press the "Print Bill "button the information about the bill appears

DIFFICULTIES

IN THE MYSQL CODE:

- **Issue1:** An error appeared when creating the database or entities, stating that they "already exist."
Solution: Use the DROP IF EXISTS statement before creating the database or entities.
- **Issue2:** We faced difficulties when adding data, such as attempting to add a branch that has a foreign key linked to a Menu, which hadn't been added yet.
Solution: Set the Menu column to NULL initially, then update the menu field after adding the Menu.

INTEGRATING MYSQL WITH NETBEANS:

- **Issue3:** We faced numerous errors and difficulties while establishing the connection between MySQL and NetBeans.
Solution: Spent an entire day researching YouTube tutorials**. (We suggest including clear instructions for integration in the project requirements).**

IN NETBEANS:

- **Issue4:** Designing and aligning the frames (GUI) took a significant amount of time due to the lack of flexibility and smoothness in Java's GUI framework.